Functions

Function calls

In Lox, a function call is simply an expression followed by a pair of parentheses

Inside the parentheses, we can pass a comma-separated list of arguments

But what exactly are we calling?

Callee

The expression immediately preceding the parentheses is called the callee

Usually, this is just an identifier looking up a function by its name

But it can be *any* expression that evaluates to something callable. This allows us to chain calls or call functions returned by other functions!

```
// Calling a named function
printValue(123);

// Calling a function returned by another function
getCallback()();
```

Syntax

Function calls have very high precedence, sitting right below primary expressions but above unary operators

Notice the `*` in the `call` rule. This allows us to chain calls indefinitely, like `fn(1)(2)(3)`

```
unary     → ( "!" | "-" ) unary | call ;
call      → primary ( "(" arguments? ")" )* ;
arguments → expression ( "," expression )* ;
```

To support this, we must update our existing `unary` rule to fall through to `call` instead of jumping straight to `primary`

AST

Because a function call has distinct components, we need a new AST node to represent it

It stores the callee expression, the closing parenthesis token (for reporting runtime errors at the correct location), and a list of argument expressions

```
tool/GenerateAst.java
in main()
```

```
    "Binary   : Expr left, Token operator, Expr right",
    "Call     : Expr callee, Token paren, List<Expr> arguments",
    "Grouping : Expr expression",
```

Parser

First, we update the `unary()` method in our parser to point to the new precedence level

```
lox/Parser.java
in unary()
```

```
    if (match(BANG, MINUS)) {
        Token operator = previous();
        Expr right = unary();
        return new Expr.Unary(operator, right);
    }

    return call(); // Changed from primary()
```

Parser (cont)

Next, we implement the `call()` method

It first parses a primary expression (the callee). Then, it enters a `while` loop

Each time it sees a left parenthesis, it parses a function call using the previously parsed expression as the callee. This loop is what handles chained function calls!

```
lox/Parser.java
after unary()
```

```
private Expr call() {
    Expr expr = primary();
    while (true) {
        if (match(LEFT_PAREN)) {
            expr = finishCall(expr);
        } else {
            break;
        }
    }
    return expr;
}
```

Maximum argment counts

We can technically parse an unlimited number of arguments

However, the C implementation of Lox (clox) that we'll build later has a hard limit of 255 arguments due to how its bytecode instructions are packed

To ensure our Java interpreter is fully compatible with our future C interpreter, we enforce the same limit here during parsing

```
lox/Parser.java
in finishCall()
```

```
        do {
            if (arguments.size() ≥ 255) {
                error(peek(), "Can't have more than 255 arguments.");
            }
        }
```

Parser

Finally, we parse the arguments inside the parentheses in `finishCall()`

We keep parsing expressions separated by commas until we hit the closing parenthesis

Parser (cont)

```
lox/Parser.java
after call()
```

```
private Expr finishCall(Expr callee) {
    List<Expr> arguments = new ArrayList<>();
    if (!check(RIGHT_PAREN)) {
        do {
            if (arguments.size() >= 255) {
                error(peek(), "Can't have more than 255 arguments.");
            }
            arguments.add(expression());
        } while (match(COMMA));
    }

    Token paren = consume(RIGHT_PAREN, "Expect ')' after arguments.");

    return new Expr.Call(callee, paren, arguments);
}
```

Interpreting function calls

When we evaluate a call expression, we first evaluate the callee expression

Then we evaluate each of the argument expressions in order and store them in a list

```
lox/Interpreter.java
in class Interpreter
```

```
@Override
public Object visitCallExpr(Expr.Call expr) {
    Object callee = evaluate(expr.callee);

    List<Object> arguments = new ArrayList<>();
    for (Expr argument : expr.arguments) {
        arguments.add(evaluate(argument));
    }
```

Interpreting function calls (cont)

Once we know the callee is callable, we cast it to our `LoxCallable` interface

Then we simply invoke its `call()` method, passing in the interpreter and the arguments we evaluated earlier!

```
lox/Interpreter.java
in visitCallExpr()
```

```
    LoxCallable function = (LoxCallable)callee;
    // (arity check goes here)
    return function.call(this, arguments);
}
```

Lox callable interface

Not all Lox objects can be called. A string isn't callable, and neither is a number

We need a way in Java to represent a Lox object that can be called like a function

```
lox/LoxCallable.java
create new file
```

```
package com.craftinginterpreters.lox;

import java.util.List;

interface LoxCallable {
    Object call(Interpreter interpreter, List<Object> arguments);
}
```

Call type errors

What happens if the callee isn't actually a function? Like a string or a number?

We need to check that the evaluated callee is something we can actually call. We do this by checking if it implements a new `LoxCallable` interface

```
lox/Interpreter.java
in visitCallExpr()
```

```
    if (!(callee instanceof LoxCallable)) {
        throw new RuntimeError(expr.paren,
            "Can only call functions and classes.");
    }

    LoxCallable function = (LoxCallable)callee;
```

Checking arity

Before we execute the call, we must ensure the caller provided the exact number of arguments the function expects

This expected number of arguments is called the function's arity

```
lox/Interpreter.java
in visitCallExpr(), before function.call()
```

```
    if (arguments.size() ≠ function.arity()) {
        throw new RuntimeError(expr.paren, "Expected " +
            function.arity() + " arguments but got " +
            arguments.size() + ".");
    }
```

```
lox/LoxCallable.java
in interface LoxCallable
```

```
interface LoxCallable {
    int arity();
```

Native Functions

Interacting with the Outside World

A programming language that can only manipulate its own internal state is fundamentally useless. Without a way to interact with the outside world, a script cannot read input, write output, check the time, or access the file system

To solve this, we expose functions to the user that are implemented in the underlying implementation language (in our case, Java) rather than in Lox itself

These are called native functions. You may also hear them referred to as *primitives*, *external functions*, or *foreign functions* (via a Foreign Function Interface, or FFI)

The Global Environment

Before we define any native functions, we need a place to put them so the user's scripts can access them

We need a fixed reference to the outermost global environment. Currently, our `Interpreter` class just has an `environment` field that changes as it enters and exits local blocks

We add a permanent `globals` field to hold our top-level bindings

lox/Interpreter.java
in class Interpreter

```
    final Environment globals = new Environment();
    private Environment environment = globals;
```

Injecting Built-ins

We want native functions to be available as soon as the interpreter starts, *before* any user code executes

To do this, we inject them directly into the `globals` environment inside the `Interpreter`'s constructor

Our first native function will be `clock()`, which returns the current time. This is essential for users to benchmark their Lox programs

```
lox/Interpreter.java
in class Interpreter
```

```
    Interpreter() {
        globals.define("clock", new LoxCallable() {
            // Implementation...
        });
    }
```

Implementing `clock`

We define the native function by passing an anonymous class that implements our `LoxCallable` interface

The `arity()` method returns `0` because `clock` takes no arguments

The `call()` method fetches the system time using Java's `System.currentTimeMillis()`. It divides by `1000.0` to convert the milliseconds to seconds and returns it as a `Double`, which is Lox's internal representation for all numbers

```
lox/Interpreter.java
inside the anonymous LoxCallable class
```

```
            @Override
            public int arity() { return 0; }

            @Override
            public Object call(Interpreter interpreter,
                               List<Object> arguments) {
                return (double)System.currentTimeMillis() / 1000.0;
            }
```

String Representation

There is one minor edge case to handle. What happens if a user writes `print clock;`?

Because `clock` is an object in our environment, the interpreter will evaluate it and pass the anonymous Java object to our `stringify()` method

By default, Java will print a cryptic memory address like `com.craftinginterpreters.lox.Interpreter$1@7150bd4d`

To fix this and provide a polished user experience, we override `toString()` directly on the anonymous class

```
1  lox/Interpreter.java
2  inside the anonymous LoxCallable class
```

```
1              @Override
2              public String toString() { return "<native fn>"; }
```

Function Declarations

Custom Functions

We have function calls and native functions, but we still can’t define our own custom Lox functions

To do that, we need to introduce a new statement to our language for function declarations

In Lox, we declare functions using the `fun` keyword, followed by a name, a list of parameters, and a block body

A New AST Node

A function declaration binds together a name, a list of parameters, and the body of the function

Because it binds a name to a value, it is a declaration rather than a simple statement

We add a new `Function` node to our `Stmt` class

```
tool/GenerateAst.java
in main() under Stmt
```

```
    "Expression : Expr expression",
    "Function   : Token name, List<Token> params, List<Stmt> body",
    "If         : Expr condition, Stmt thenBranch, Stmt elseBranch",
```

Updating the Grammar

We add function declarations to our grammar at the `declaration` level

We separate the `function` rule itself from `funDecl` so that we can reuse the `function` rule later when we implement methods in classes

```
declaration → funDecl
            | varDecl
            | statement ;

funDecl     → "fun" function ;
function    → IDENTIFIER "(" parameters? ")" block ;
parameters  → IDENTIFIER ( "," IDENTIFIER )* ;
```

Hooking into the Parser

We update our `declaration()` method to look for the `fun` keyword

```
lox/Parser.java
in declaration()
```

```
private Stmt declaration() {
    try {
        if (match(FUN)) return function("function");
        if (match(VAR)) return varDeclaration();
}
```

Parsing the Function

The `function()` method parses the name, the parameters, and the body

We pass in a `kind` string ("function" or "method") so we can reuse this exact same parsing logic for class methods later, and provide accurate error messages!

```
lox/Parser.java
after varDeclaration()
```

```
private Stmt.Function function(String kind) {
    Token name = consume(IDENTIFIER, "Expect " + kind + " name.");
    consume(LEFT_PAREN, "Expect '(' after " + kind + " name.");

    List<Token> parameters = new ArrayList<>();
    // ... parameter parsing goes here ...

    consume(RIGHT_PAREN, "Expect ')' after parameters.");
    // ... body parsing goes here ...
}
```

Parsing Parameters

Inside the parentheses, we parse the parameter list

Just like with function calls, we enforce a maximum limit of 255 parameters to remain compatible with our future C-based bytecode interpreter

```
lox/Parser.java
inside function()
```

```
    if (!check(RIGHT_PAREN)) {
        do {
            if (parameters.size() ≥ 255) {
                error(peek(), "Can't have more than 255 parameters.");
            }

            parameters.add(
                consume(IDENTIFIER, "Expect parameter name."));
        } while (match(COMMA));
    }
```

Parsing the Body

Finally, we parse the body of the function

The body of a function is always a block, so we consume the opening brace and then call our existing `block()` parsing method

We wrap it all up into our new `Stmt.Function` AST node

lox/Parser.java
inside function(), continued

```
        consume(LEFT_BRACE, "Expect '{' before " + kind + " body.");
        List<Stmt> body = block();
        return new Stmt.Function(name, parameters, body);
    }
```